



# PARLIA ELC Security Review



JANUARY, 2026

[www.chaindefenders.xyz](http://www.chaindefenders.xyz)  
<https://x.com/DefendersAudits>

## Lead Auditors



PeterSR



0x539.eth

## Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Commit Hash
  - Scope
- Executive Summary
  - Issues Found
  - Security Grade
  - Audit Overview
- Findings
  - Low
  - Informational

## Protocol Summary

Parlia ELC is a light client for BNB Smart Chain (BSC) that utilizes the Parlia consensus mechanism. It serves as a Light Client (ELC) implementation that enables secure cross-chain communication between BSC and other blockchains without requiring a full node. This light client integrates with the Inter-Blockchain Communication (IBC) protocol framework to facilitate trustless verification of BSC state and data.

BNB Smart Chain uses Parlia, a Proof-of-Staked-Authority (PoSA) consensus engine derived from Ethereum's Clique consensus with additional modifications such as:

- Epoch-based validator rotation
- Stake-weighted validator selection
- Fast block times
- Limited validator set size

## Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

## Risk Classification

| Likelihood/Impact | High | Medium | Low |
|-------------------|------|--------|-----|
| High              | H    | H/M    | M   |
| Medium            | H/M  | M      | M/L |
| Low               | M    | M/L    | L   |

## Audit Details

Commit Hash

610ab92853307ee0c8171575fd4055f8166c09e3

## Scope

| Id | Files in scope                              |
|----|---------------------------------------------|
| 1  | proto/src/prost/google.protobuf.rs          |
| 2  | light-client/build.rs                       |
| 3  | light-client/src/message.rs                 |
| 4  | light-client/src/consensus_state.rs         |
| 5  | light-client/src/commitment.rs              |
| 6  | light-client/src/client_state.rs            |
| 7  | light-client/src/client.rs                  |
| 8  | light-client/src/lib.rs                     |
| 9  | light-client/src/misc.rs                    |
| 10 | light-client/src/misbehaviour.rs            |
| 11 | light-client/src/errors.rs                  |
| 12 | light-client/src/fork_spec.rs               |
| 13 | light-client/src/header/vote_attestation.rs |
| 14 | light-client/src/header/eth_headers.rs      |
| 15 | light-client/src/header/eth_header.rs       |
| 16 | light-client/src/header/mod.rs              |
| 17 | light-client/src/header/constant.rs         |
| 18 | light-client/src/header/validator_set.rs    |
| 19 | light-client/src/header/epoch.rs            |

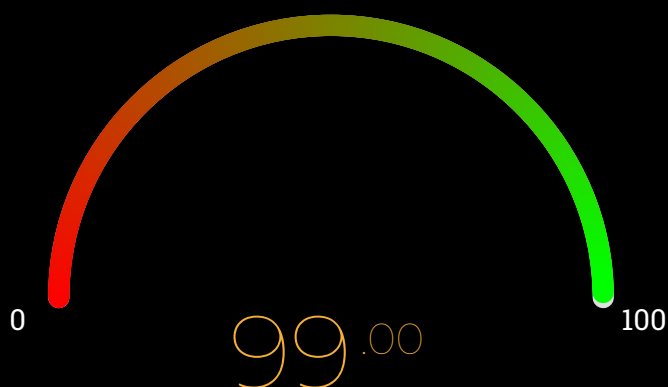
## Executive Summary

## Issues Found

| Severity | Count | Description                  |
|----------|-------|------------------------------|
| High     | 0     | Critical vulnerabilities     |
| Medium   | 0     | Significant risks            |
| Low      | 2     | Minor issues with low impact |

| Severity      | Count | Description                   |
|---------------|-------|-------------------------------|
| Informational | 1     | Best practices or suggestions |
| Gas           | 0     | Optimization opportunities    |

## Security Grade



Based on the audit findings and code quality, the overall protocol security grade is **99.00 / 100.00**. This reflects an excellent security posture with almost no room for improvement.

## Audit Overview

This audit provides a comprehensive evaluation of the Parlia Light Client (ELC) to certify its readiness and compliance with the Fermi Hard Fork on the BNB Smart Chain (BSC). The primary objective was to ensure that the light client's verification predicates remain robust and accurate following the implementation of the new BNB Evolution Proposals (BEPs).

## Scope of Compliance

The audit verified that the Parlia ELC logic is fully compatible with the following BEPs included in the Fermi upgrade:

- BEP-590 (Extended Voting Rules for Fast Finality Stability)
- BEP-619 (Short Block Interval Phase Three (0.45 seconds))
- BEP-592 (Non-Consensus Based Block-Level Access List)
- BEP-593 (Incremental Snapshot)
- BEP-610 (Implement EVM Super Instruction)

## Methodology & Audit Path

The audit was conducted through a systematic review of the light client's specification and its implementation of the Parlia consensus protocol. The process was divided into the following functional stages:

**Architectural Differential Analysis** The audit began by mapping all structural changes introduced by the Fermi hard fork against the existing Parlia ELC codebase. We performed a side-by-side analysis of the legacy version versus the Fermi-compatible version to identify critical differences in state management and header field processing. This helped anticipate potential pitfalls.

**Attestation & Finality Logic Verification** A significant portion of the audit was dedicated to the core verification functions:

- `verify_target_attestation` & `verify_vote_attestation`: These were scrutinized to ensure they correctly interpret the attestation and finality rules defined in BEP-590.
- `verify_finalized`: We confirmed that the logic for identifying a finalized header is valid under the new consensus parameters.

**Header Validation & BEP Alignment** The audit team performed a granular review of the `verifyHeader` function to ensure there was no regression in its functionality.

**Final Review & Report Generation** The final phase involved a comprehensive regression check. We re-validated every protocol change against the technical specifications of the five core BEPs. This ensured that the client remains secure and functional in the post-Fermi environment.

## Conclusion

The audit concludes that the Parlia ELC is fully compliant with the Fermi Hard Fork. The client successfully implements the necessary logic.

## Findings

### Low

#### Low 01 Misleading Docstring And Comment Regarding Target Block In `verify_target_attestation`

### Location

`eth_header.rs`

### Description

The docstring and comment for the `verify_target_attestation` function in `eth_header.rs` is inaccurate. The function documentation states:

“This function checks the target vote attestation of the current header to ensure that the target block is the direct parent of the current block.”

A comment in the function also states:

“The target block should be direct parent.”

However, the function does not actually require the target to be a direct parent. Instead, it only verifies that the target block number and hash match the provided parent header’s number and hash. Since the Fermi hard fork allows skipped blocks, the parent passed to this function may not be the direct predecessor, but rather any ancestor block.

Leaving these confusing comments can lead to wrong assumptions and faulty usage.

## Proof of Concept

### The function signature and implementation:

```
1 pub fn verify_target_attestation(&self, parent: &ETHHeader) -> Result<
    VoteAttestation, Error> {
2     let target_vote_attestation = self.get_vote_attestation()?;
3     let target_data = &target_vote_attestation.data;
4
5     // The target block should be direct parent.
6     if target_data.target_number != parent.number || target_data.
       target_hash != parent.hash {
7         return Err(Error::UnexpectedTargetVoteAttestationRelation( ... )
8     );
9     }
10    Ok(target_vote_attestation)
```

The function only checks that `target_data` matches the provided `parent` parameter. It does not validate that `parent.number == self.number - 1`, which would be required to ensure a direct parent relationship. The Fermi hard fork test cases in `eth_headers.rs` demonstrate scenarios where blocks have skipped numbers (e.g., blocks 506, 507, 508, 509, 510 with non-consecutive target relationships).

## Recommendation

### Update the docstring to accurately reflect the function's behavior:

```
1 /// Verifies the target attestation of the current `ETHHeader` against
   a given parent header.
2 ///
3 /// This function checks the target vote attestation of the current
   header to ensure that
4 /// the target block matches the provided parent header (by both
   number and hash).
5 /// The parent does not need to be the direct predecessor; the Fermi
   hard fork allows skipped blocks.
6 pub fn verify_target_attestation(&self, parent: &ETHHeader) -> Result<
    VoteAttestation, Error> {
```

Also remove the wrong comment:

```
1 /// Verifies the target attestation of the current `ETHHeader` against
  a given parent header.
2 ///
3 /// This function checks the target vote attestation of the current
  header to ensure that
4 /// the target block matches the provided parent header (by both
  number and hash).
5 /// The parent does not need to be the direct predecessor; the Fermi
  hard fork allows skipped blocks.
6 pub fn verify_target_attestation(&self, parent: &ETHHeader) -> Result<
  VoteAttestation, Error> {
7     let target_vote_attestation = self.get_vote_attestation()?;
8     let target_data = &target_vote_attestation.data;
9
10    if target_data.target_number != parent.number || target_data.
      target_hash != parent.hash {
11        return Err(Error::UnexpectedTargetVoteAttestationRelation( ... )
12    );
13    }
14    Ok(target_vote_attestation)
```

Status

Fixed

Low 02 **ctype** Described As Deprecated But Not Marked As Such

Location

google.protobuf.rs

Description

The `ctype` field in the `FieldOptions` struct is documented as deprecated in the comment but is not formally marked with the `#[deprecated]` attribute. The inline documentation states:

“NOTE: ctype is deprecated. Use `features.(pb.cpp).string_type` instead.”

and includes a TODO comment:

“TODO: make ctype actually deprecated.”

However, the field definition lacks the Rust `#[deprecated]` attribute that would:

- Generate compiler warnings when the field is used
- Signal to IDE tooling that this field should be avoided
- Provide a clear deprecation path for users
- Align with Rust best practices for marking deprecated items

This inconsistency between the documentation stating deprecation and the lack of a formal deprecation marker means that code using `ctype` will not receive any compiler or tooling warnings, potentially leading to continued use of a deprecated feature.

## Recommendation

Add the `#[deprecated]` attribute to the `ctype` field definition with a message directing users to the replacement:

```
1  #[prost(
2      enumeration = "field_options::CType",
3      optional,
4      tag = "1",
5      default = "String"
6  )]
7  #[deprecated = "Use `features.(pb.cpp).string_type` instead"]
8  pub ctype: ::core::option::Option<i32>,
```

This will ensure that Rust’s compiler and IDE tooling properly warn users about the deprecation and encourage migration to the recommended alternative.

## Status

### Fixed

Response: This issue is within the code auto-generated by Protobuf, and the output varies depending on the Protobuf version used. Version is now publicly stated in the README.

## Informational

### Info 01 Finish Clarifying `java_string_check_utf8` And Update Docs Accordingly

Location

`google.protobuf.rs`

Description

The documentation for the `java_string_check_utf8` option in the `FileOptions` struct is incomplete and contains a TODO comment indicating that the behavior needs clarification. The current documentation states:

“A proto2 file can set this to true to opt in to UTF-8 checking for Java, which will throw an exception if invalid UTF-8 is parsed from the wire or assigned to a string field.”

However, the inline TODO comment explicitly notes:

“TODO: clarify exactly what kinds of field types this option applies to, and update these docs accordingly.”

The documentation also mentions behavior specific to Proto3 files:

“Proto3 files already perform these checks. Setting the option explicitly to false has no effect: it cannot be used to opt proto3 files out of UTF-8 checks.”

But the scope and exact applicability of this option across different field types (e.g., repeated fields, message fields, etc.) and protocol versions remains unclear and unspecified in the documentation.

Recommendation

Complete the TODO by clarifying and updating the documentation accordingly.

Status

Fixed

Response: This issue is within the code auto-generated by Protobuf, and the output varies depending on the Protobuf version used. Version is now publicly stated in the README.